Retrieval-Augmented Generation (RAG)

- Module 07: Grounding LLMs to Cure Hallucinations

- Theme: "*Giving the Model an Open Book: Accuracy through Context*.“
- 2025+ it is “Context Engineering”


MODULE 07 | 2026 EDITION - AGENTIC RAG ARCHITECTURE - "From Retrieval to Reasoning: The Autonomous Loop" - THEME BADGE: "Taming the Stochastic Parrot"
2020 Naive RAG (Retrieve & Append)
2022 Dense RAG (Embeddings)
2024 Advanced RAG (Re-ranking + Fusion)
2026 AGENTIC RAG (Autonomous Reasoning)
AGENTIC RAG ORCHESTRATOR
QUERY PLANNER (Intent Router)
SELF-REFLECTION (Verification)
CONTEXT COMPRESSOR (Rerank+Filter)
RETRIEVAL
VECTOR DB
GRAPH KNOWLEDGE BASE
KEYWORD SEARCH
HYBRID FUSION
LIVE APIs
Query: Latest advancements in quantum computing... Retrieving... Analyzing... Generating response...
GENERATION CONTROLLER (Temp/Top-p/Beam)
LLM + LoRA ADAPTERS
GROUNDED OUTPUT
+ Citations
+ Confidence
THE AGENTIC REASONING LOOP
THINK
PLAN
RETRIEVE
REFLECT
KEY INSIGHT: Agents can DECIDE to retrieve more, rephrase queries, or verify outputs AUTONOMOUSLY before final generation.
GENERATION CONTROL PANEL
TEMPERATURE
TOP-P
(Nucleus, 0.1 to 1.0)
TOP-K
(1 to 100)
BEAM SEARCH
VS
NUCLEUS
Factual Q&A
Stochastic, Creative
EFFICIENCY LAYER: LoRA + QLoRA (2026 Standard)
$W_{new} = W_{frozen} + (B \times A)$
FROZEN WEIGHTS (BB)
TRAINABLE (0.5B%)
A
B

The Two Failures of Frozen Models

1. Knowledge Cutoff:

Scenario: "Who won the World Series in 2025?"

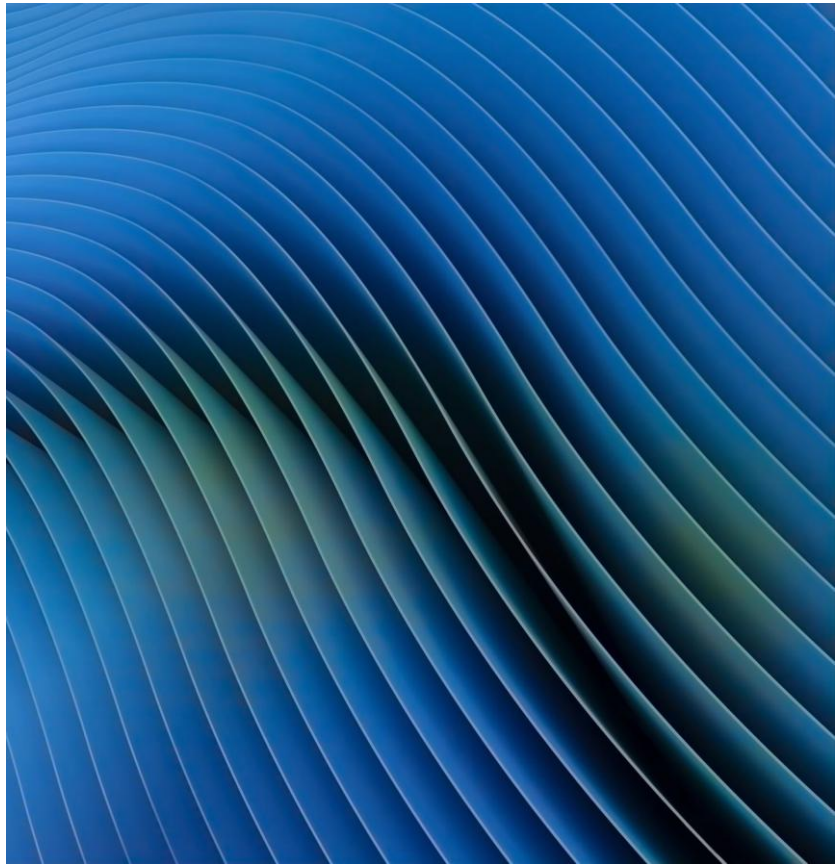
Model: "I do not know" or guesses based on old probabilities.

2. Hallucination:

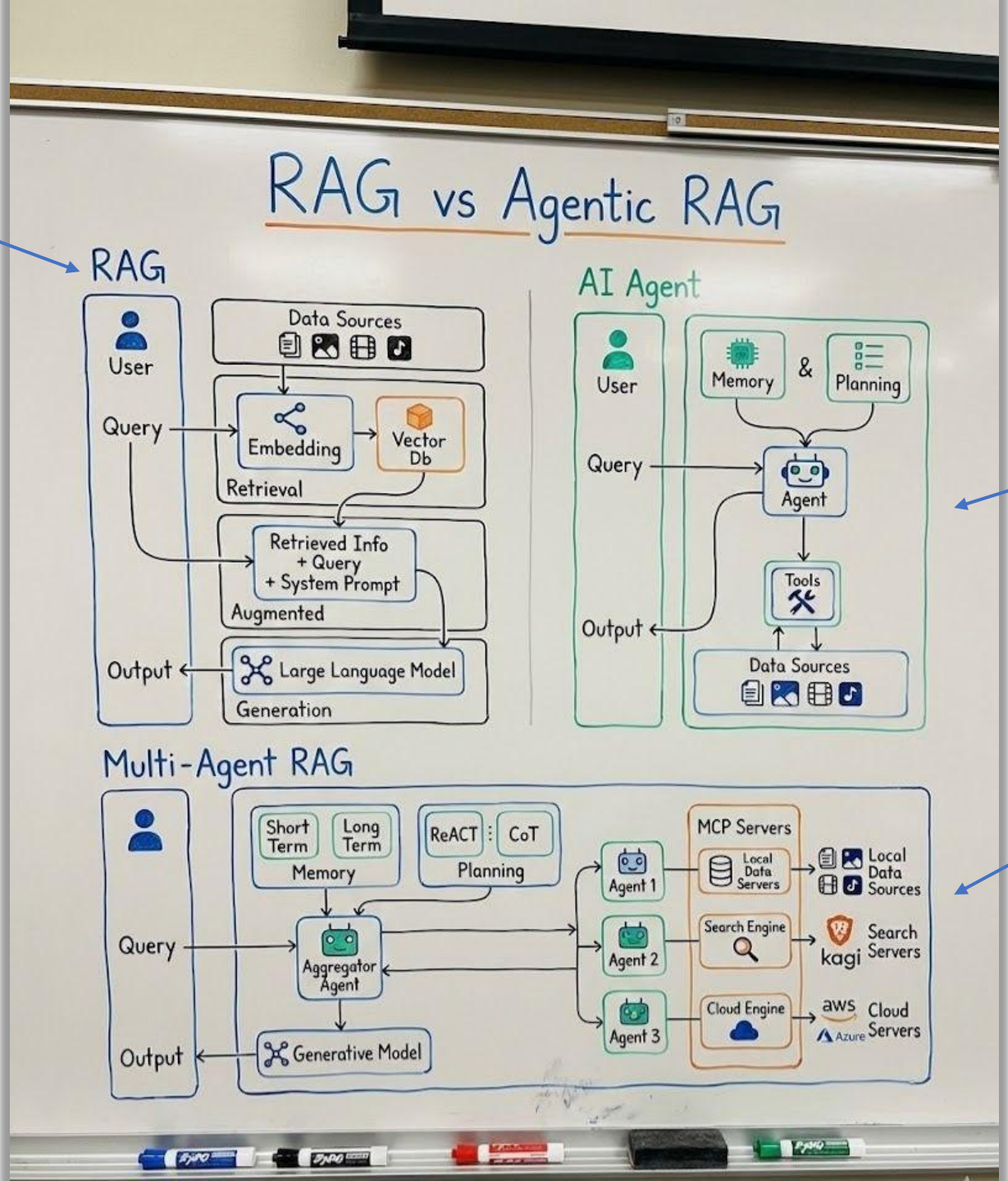
Scenario: "Summarize the safety manual for this specific robot arm."

Model: Invents a plausible but dangerous procedure.

The RAG Solution: Don't ask to memorize. Ask to process provided facts.



*You are here



Enterprise in 2025

2026

MODULE 07

RETRIEVAL-AUGMENTED GENERATION (RAG): 2025-2026

GIVING THE MODEL AN OPEN BOOK

*Don't ask the model to memorize. Ask it to **process what you provide.***

THE FROZEN MODEL PROBLEM

KNOWLEDGE CUTOFF

- Trained in 2023

"Who won 2025 World Series?"

" I don't know..."

HALLUCINATION

"Summarize this robot's manual"

DANGER!

THE RAG TRIAD: INDEX → RETRIEVE > GENERATE

INDEXER

500+ Page Manuals + Smart Chunks

Split • Embed • Store

RETRIEVER

Top-K: 3-5 Optimal Chunks

GENERATOR

Context + Prompt = Verified Answer

AGENTIC AI 2026: THE FUTURE

- 2024: *Basic RAG*
- 2025: Advanced RAG
- 2026: Multi-Agent *Context Systems*

RAG = OPEN BOOK EXAM FOR AI

MEMORIZE & GUESS → C?

LOOKUP & VERIFY → A+

STUDENT TAKEAWAY 2026

RAG is *TODAY's industry standard for enterprise AI.*

TOMORROW's Agentic AI will automatically engineer its own context—deciding what to retrivve, when to search, and how to verify.

Master RAG now. Engineer agents next.

*The future belongs to **CONTEXT ENGINEERS.***

LAB: *Build CourseBot* → **Your Data + LLM Reasoning = *Grounded Answers*** → Module 7 of 12 ▶ **Next:** *Chatbots & Agents* ❯

The "Open Book Exam" Analogy

Standard LLM (Fine-Tuning):

Student memorizes textbook, throws it away, takes test from memory.

Risk: Memory is fuzzy.

RAG:

Student takes test with textbook open on the desk.

Benefit: Look up exact page, read, synthesize.

Result: High accuracy, verifiable sources.

The Architecture High-Level

The Triad:

1. **Indexer:** Prepares your private data (PDFs, Wikis, Databases).

2. **Retriever:** Finds the most relevant "chunks" based on user question.

3. **Generator:** The LLM takes chunks + question and writes the answer.

Step 1: Ingestion & Chunking (The Art)

Why Chunk? Context Window limits & Cost. Need small, relevant pieces.

Strategies:

Fixed Size: Every 500 chars. (Simple, breaks sentences).

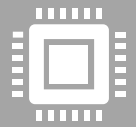
Recursive: Split by paragraphs, then sentences. (Better context).

Semantic Chunking (2025 Standard): Break when topic changes (embedding shifts).

Think of it as Long doc being sliced into overlapping tiles

Step 2: Embedding the Chunks

Recall Module 2: Convert text to Vectors.



The Process:

Pass every chunk through an Embedding Model.

e.g., OpenAI text-embedding-3-small or Hugging Face all-MiniLM-L6-v2.

Result: A list of floating-point numbers representing the meaning.

Step 3: The Vector Database

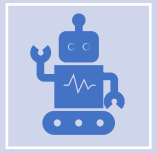
What is it? DB optimized for similarity search of vectors.

The Landscape:

Azure AI Search: Microsoft, Postgres pgvector, Pinecone: Managed, scalable, cloud-native.

ChromaDB / FAISS: Open-source, runs locally (Our Lab).

Weaviate / Qdrant: Feature-rich, hybrid search.



Robotics Context: The robot stores its map and object definitions here.

Refer: Vector Big Picture – Google Drive

Indexing Visualization

- **Flow :**
  - [Document] -> [Splitter] -> [Chunks] -> [Embedding Model] -> [Vector DB]
  - Each step represents a data transformation from raw text to mathematical understanding.

- Refer: https://thedataquarry.com/blog/vector-db-1/

Retrieval: Finding the Needle

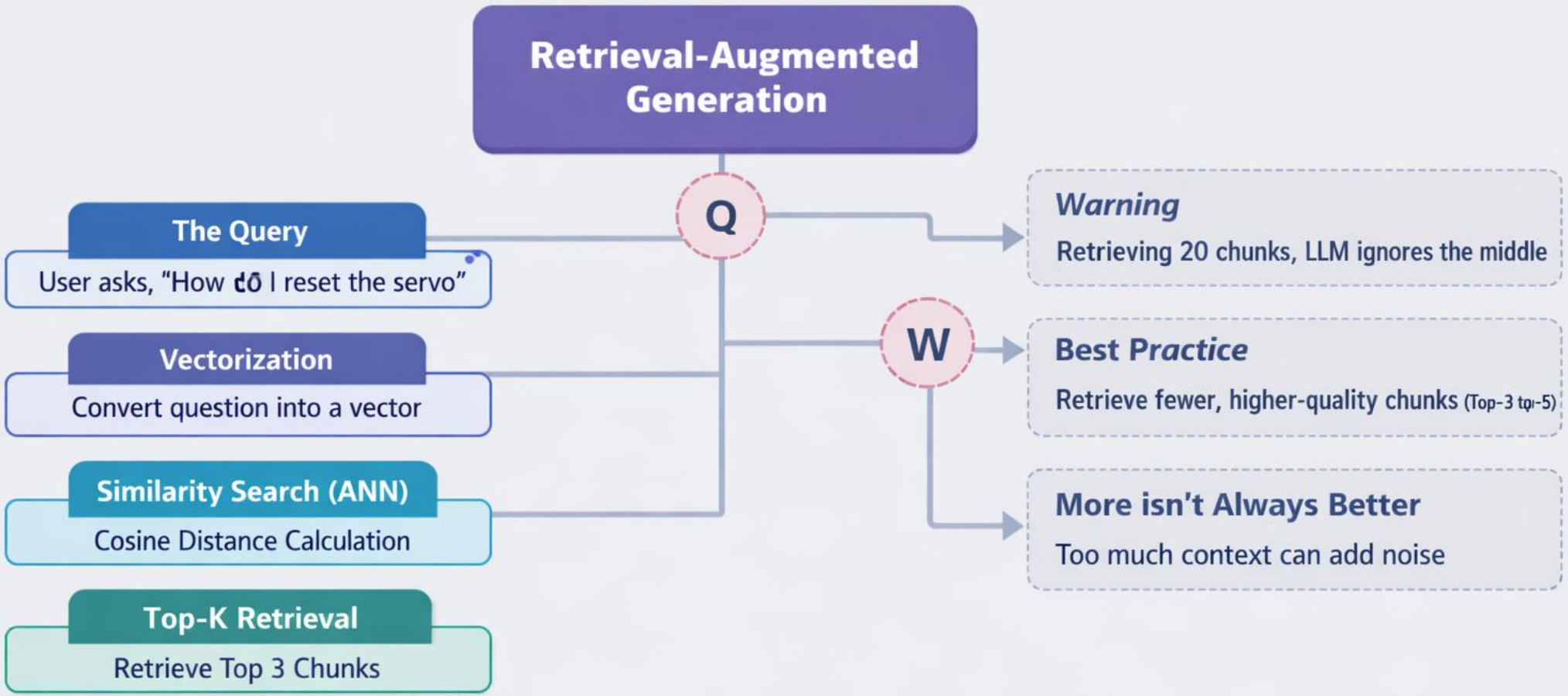
- **The Query:** User asks, "How do I reset the servo?"
- **Vectorization:** Convert question into a vector.
- **Similarity Search (ANN):** DB calculates distance (Cosine Similarity) between Question and Stored Chunks.
- **Top-K:** Retrieve the top 3 closest chunks.

- Refer: https://thedataquarry.com/blog/vector-db-2/#generative-qa-chatting-with-your-data

Understanding RAG & “Lost in the Middle” Challenges

Retrieval | ANN Search | "Lost in the Middle"

Retrieval-Augmented Generation Concepts & Issues



The "Lost in the Middle" Phenomenon

- **Warning:** If you retrieve 20 chunks, the LLM often ignores the ones in the middle.
- **Best Practice:** Retrieve fewer, higher-quality chunks (Top-3 or Top-5).
- More context is not always better; it can introduce noise.

Augmentation: The Prompt Template

- **The "Magic" happens here**. We dynamically build a prompt.
-
```
System: You are a helpful assistant. Use ONLY the context
below to answer.

Context:
- [Chunk 1 Text]
- [Chunk 2 Text]
- [Chunk 3 Text]

User Question: {question}
Answer:
```

Generation & Citation

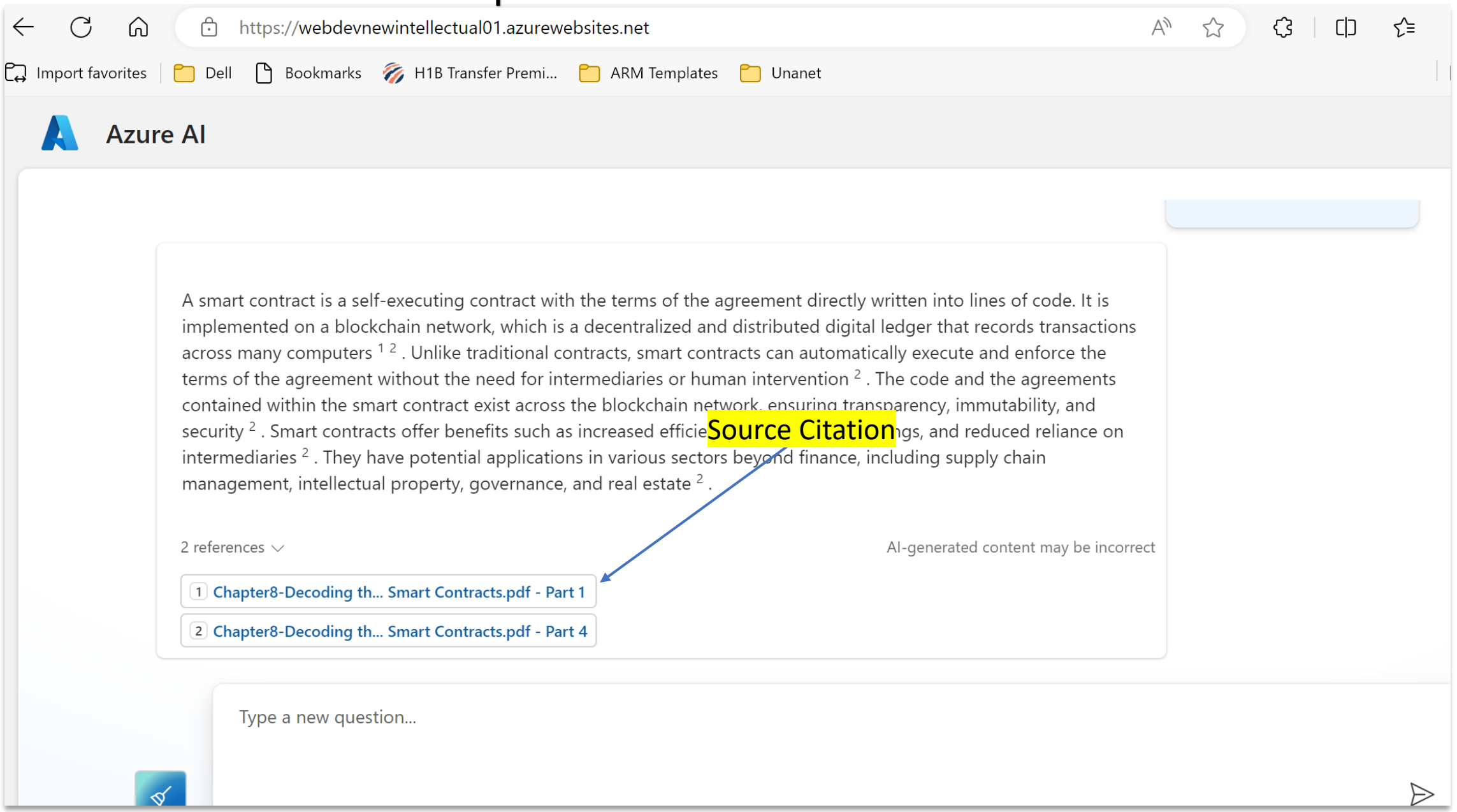
The Output: The LLM generates the answer based on provided chunks.

Verification:

- Because we know which chunks we sent, we can force citations.

Example: "Reset the servo by holding the red button (Source: Manual Pg 4)."

An Enterprise AI Search on Azure with Citations



Advanced RAG (The "World Class" Edge)

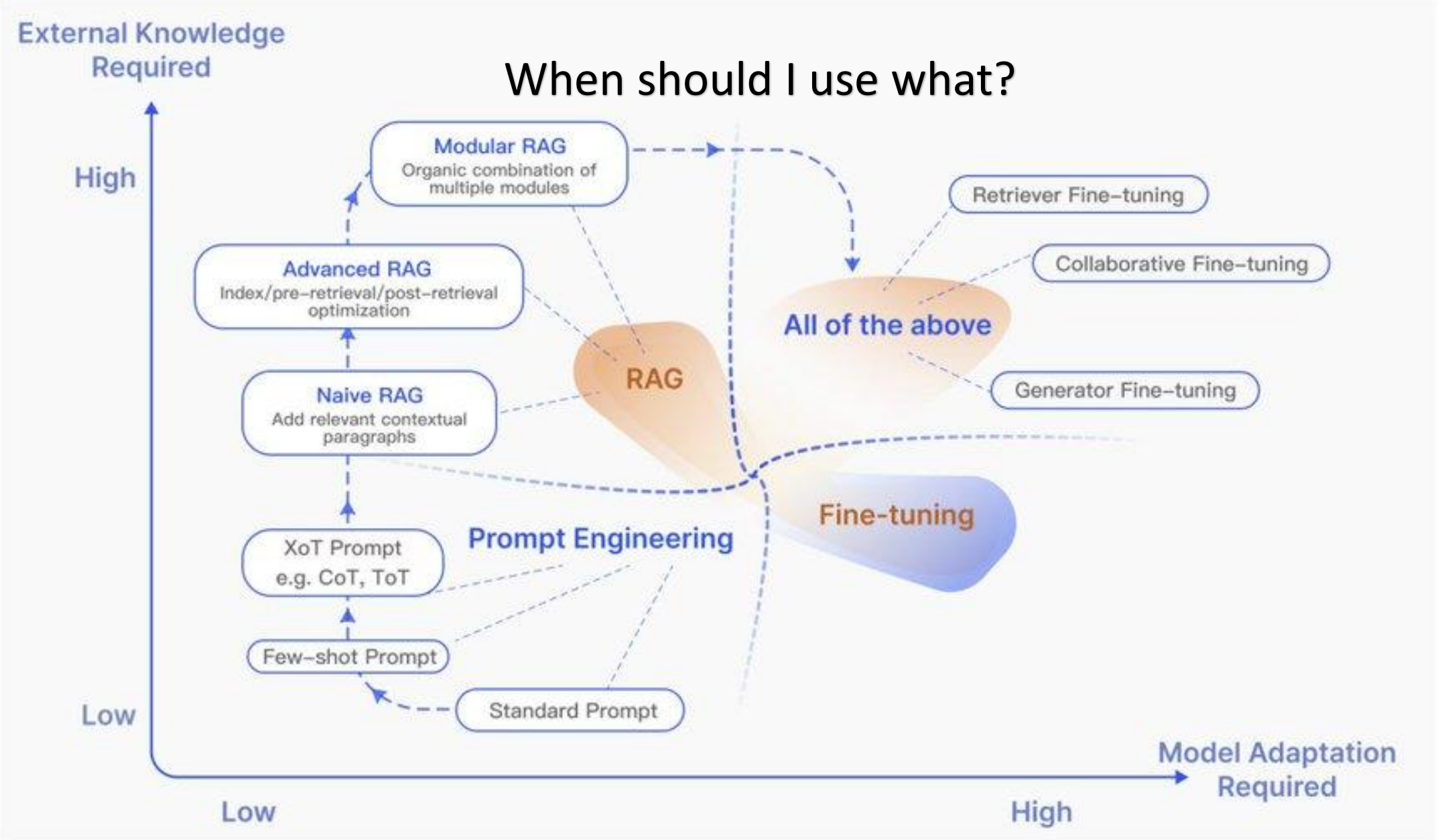
Hybrid Search:

- Vectors are bad at exact part numbers ("#XJ-9"). Keywords (BM25) are good.
- Hybrid RAG combines both.

Re-Ranking (Cross-Encoders):

- 1. Retrieve 50 documents (fast).
- 2. Use "Re-ranker" model to score against query.
- 3. Send top 5 to LLM. (Significantly boosts accuracy).

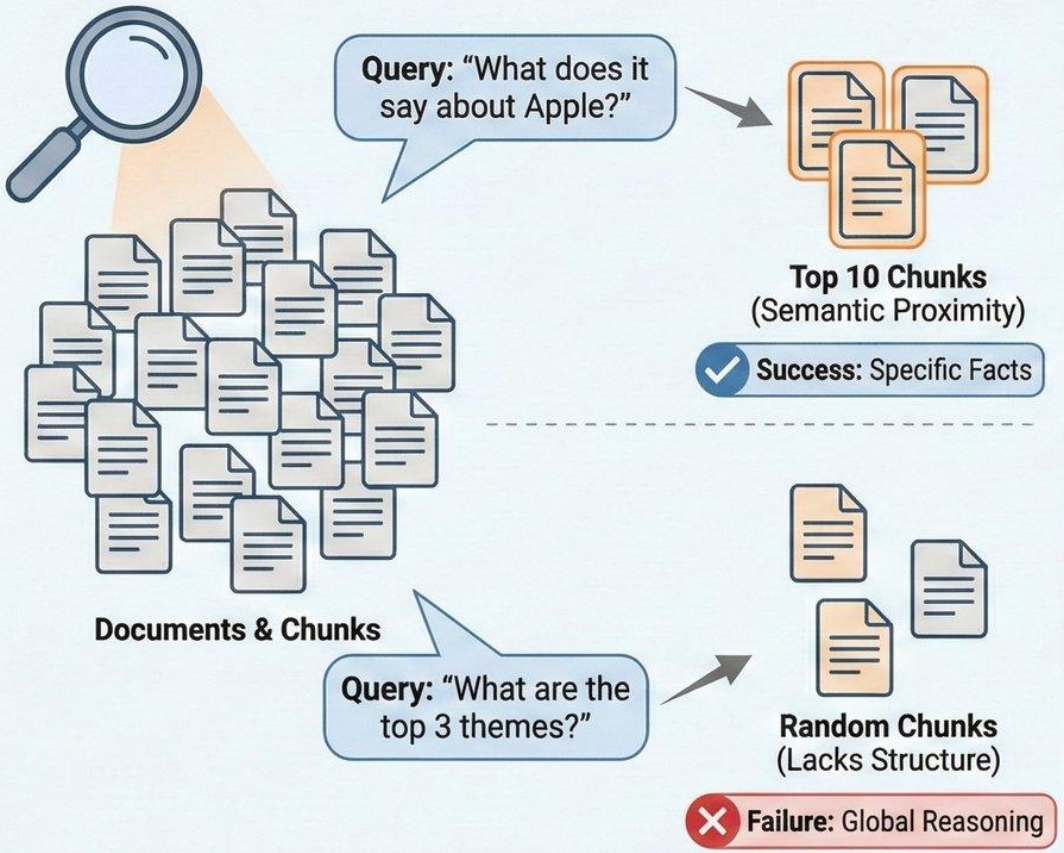
When should I use what?



Reference: https://arxiv.org/pdf/2312.10997

Vector RAG vs. GraphRAG: The Search for Global Reasoning

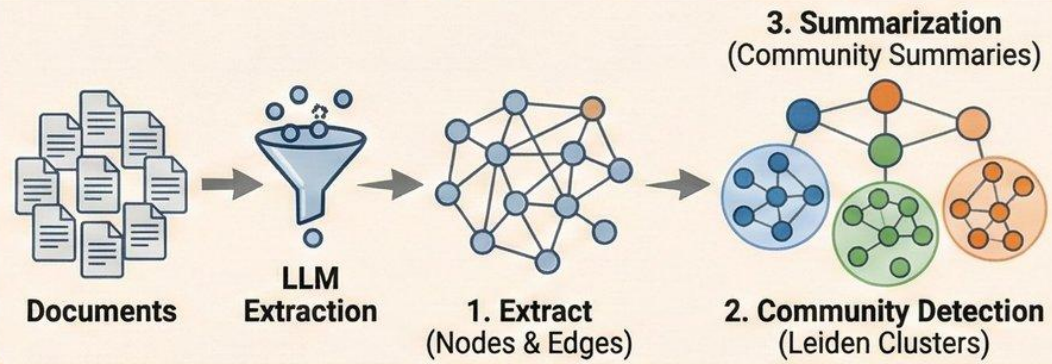
VECTOR RAG (Myopic / Similarity Search)



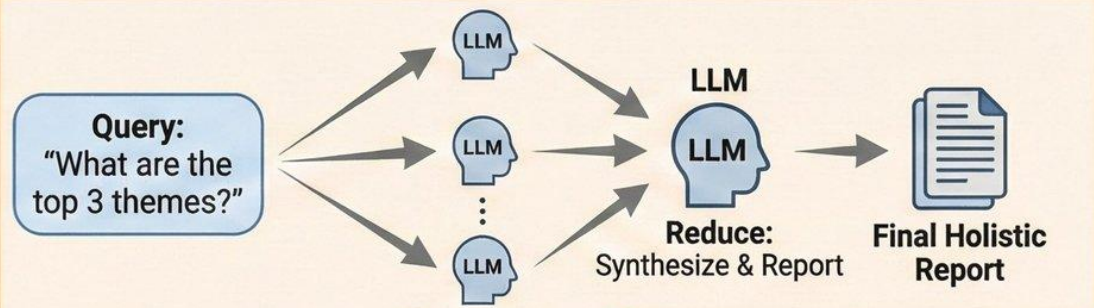
THE SHIFT: From Flat Search to Hierarchical Structure

GRAPHRAG (Holistic / Complex Reasoning)

STEP A: INDEXING (Hierarchical Knowledge Graph ETL)



STEP B: SEARCH (Map-Reduce Pattern)



Cost: Low (Embedding)
Maintenance: Simple (Re-index)

REALITY CHECK (Trade-offs)

Cost: High (10x-100x LLM Processing)
Maintenance: Complex (Noise Pruning / Re-run ETL)

SUMMARY: Use **VECTORS** for Similarity Search. Use **GRAPHS** for Complex Reasoning. Enterprise often needs **BOTH.**

Microsoft Graph RAG: https://microsoft.github.io/graphrag/

Context Engineering 2.0

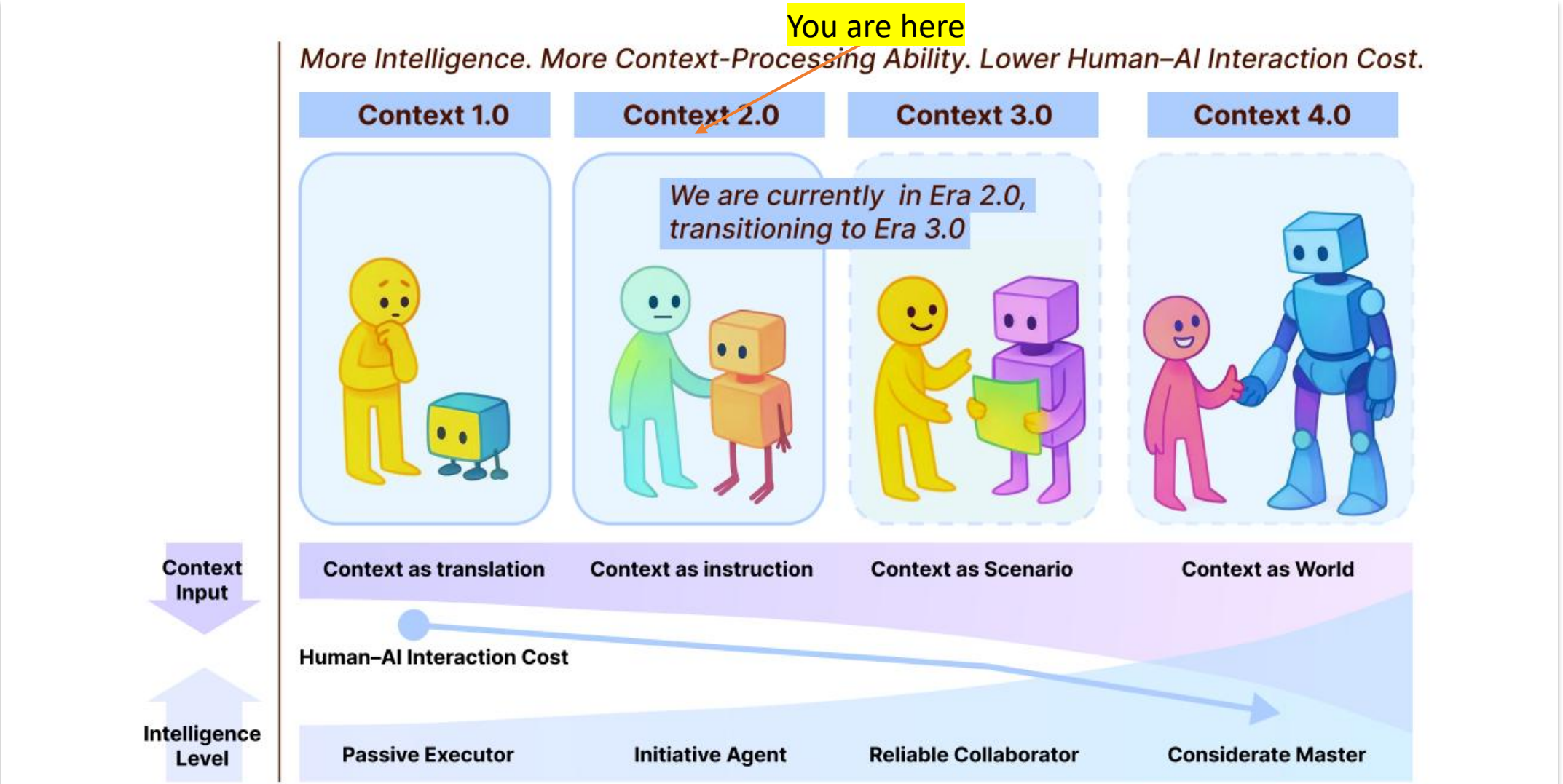


Figure 1: The Overview of context engineering 1.0 to context engineering 4.0, illustrating that more intelligence leads to greater context-processing ability and lower human-AI interaction cost.

Optional Listen: https://youtu.be/CA-QTmeAoyg

Source: https://www.arxiv.org/pdf/2510.26493

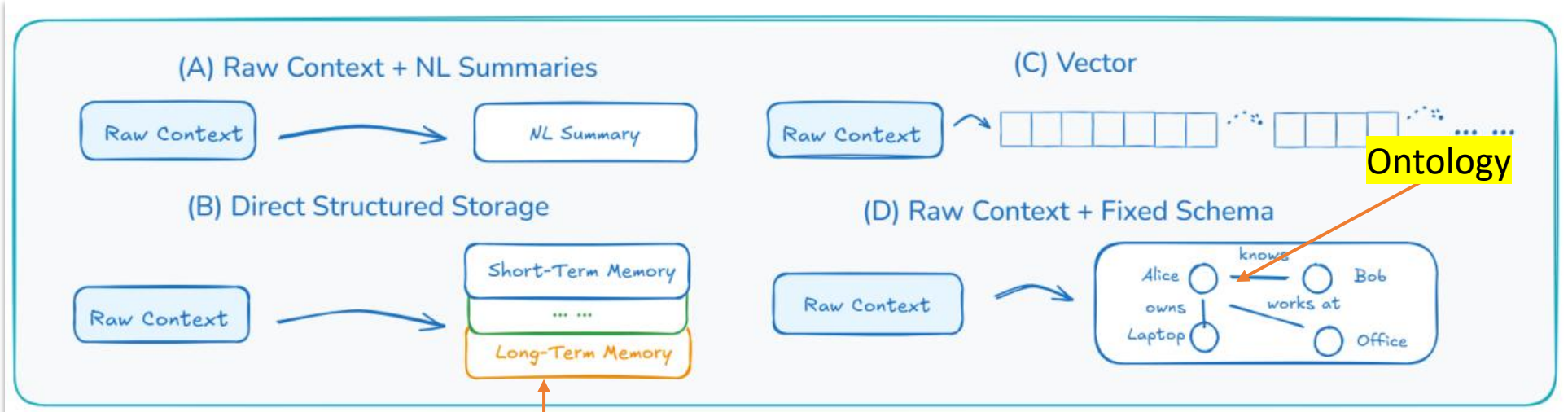


Figure 6: Representative designs for self-baking in Era 2.0

Importance of Statefulness

Source: https://www.arxiv.org/pdf/2510.26493

CONTEXT ENGINEERING 2.0

Reducing Information Entropy between Human Intent and Machine Action.

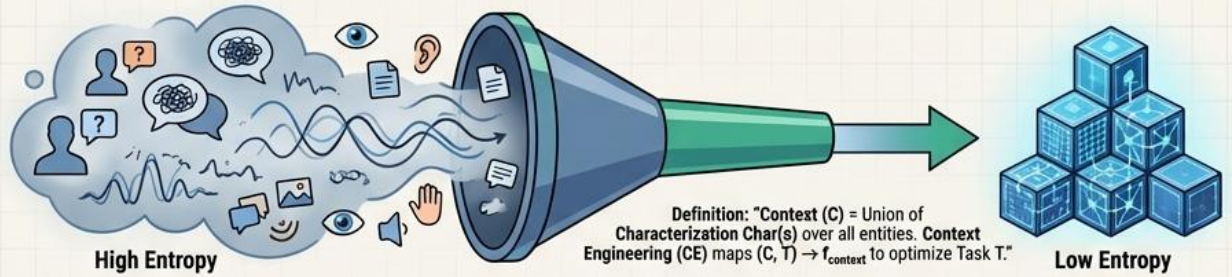
THE 4 ERAS OF EVOLUTION

Era 1.0 (1990s-2020): Primitive Computation
Sensor fusion, rule triggers. Designers as intention translators. Exemplar: Context Toolkit.

Era 2.0 (Present): Agent-Centric
Prompts, RAG, Tool use. From Context-Aware to Context-Cooperative. Interface cost falls.

Era 3.0 (Future): Human-Level
Social cues, emotion, tacit knowledge. Personal digital memory with human-like forgetting.

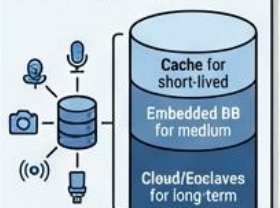
Era 4.0 (Speculative): Superhuman
God's eye view. Machines proactively construct context and reveal latent needs.



Definition: "Context (C) = Union of Characterization Char(s) over all entities. Context Engineering (CE) maps (C, T) → $f_{context}$ to optimize Task T."

Node A: Collection & Storage

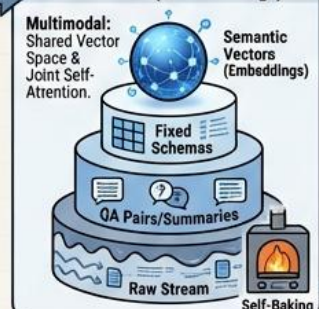
Principles: Minimal Sufficiency, Semantic Continuity.



Distributed sensors → Layered Storage

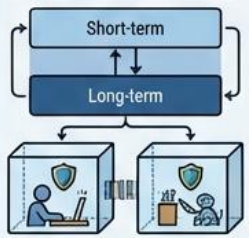
Node B: Management & Abstraction ("Self-Baking")

Multimodal: Shared Vector Space & Joint Self-Attention.



Node C: Organization & Isolation

Layered Memory (OS Analogy):



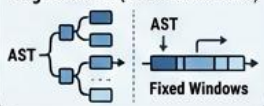
Context Isolation: Sandboxes/ Sub-agents to prevent pollution.

Node D: Selection & Retrieval

Criteria: Semantic Relevance, Logical Dependency, Recency, Deduplication.

Reranking (LLM)

Segmentation (AST vs Fixed Windows)



USAGE PATTERNS & ENGINEERING TRICKS

Card 1: Sharing & Interoperability

Intra-System:

Cross-System:

Card 2: Proactive Inference

Infer latent goals via Chain-of-Thought.

Detect 'struggle signatures' to offer tools.

Learn profiles from history.

Card 3: Emerging Engineering Practices

KV Caching: Keep prefixes stable, deterministic append-only updates.

Tooling: Small, stable toolsets. No dynamic loading (breaks cache).

Orchestration: "Todo.md" trick (recite goals in context). Lead agent decomposes tasks. Don't hide errors—use them for correction.

The Road Ahead: Solving Storage Bottlenecks, Processing Degradation (Quadratic Attention), System Instability, and Evaluation Difficulty. Goal: A Semantic Operating System.

Visual Notes for AI: Use rounded corners on all boxes. Use distinct icons for "Text" vs "Multimodal" data. Ensure text hierarchy: Headers in Bold Sans-Serif, Body text in clean regular weight. Connect the pipeline stages with thick, flowing arrow.

Lab Overview: Building "CourseBot"

Goal: Build a QA system for this course's syllabus.

The Stack: PIP Install relevant libraries

- **Orchestrator:** Use your own framework or Lang Chain (The glue code).
- **Database:** Chroma DB (Local vector store).
- **Embeddings:** Huggingface (Free, local).

LLM: OpenAI API or Google Gemini API or local ollama or LM Studio.

Step 1: Loading & Splitting

```
from langchain.document_loaders import TextLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter

# 1. Load Data
loader = TextLoader("syllabus.txt")
documents = loader.load()

# 2. Split (Chunk size 500, overlap 50 to preserve context)
text_splitter = RecursiveCharacterTextSplitter(chunk_size=500,
chunk_overlap=50)
chunks = text_splitter.split_documents(documents)
```

Step 2: Vector Store Creation

```
from langchain.vectorstores import Chroma
from langchain.embeddings import HuggingFaceEmbeddings

# 3. Create Embeddings & Store
embedding_model = HuggingFaceEmbeddings(model_name="sentence-transformers/all-
MiniLM-L6-v2")

# This creates a local folder 'db' with the vectors
vector_db = Chroma.from_documents(chunks, embedding_model,
persist_directory="./db")
```

Step 3: The RAG Chain

```
from langchain.chains import RetrievalQA
from langchain.llms import OpenAI

# 4. Setup Retrieval
retriever = vector_db.as_retriever(search_kwargs={"k": 3})

# 5. Connect to LLM
qa_chain = RetrievalQA.from_chain_type(
    llm=OpenAI(),
    chain_type="stuff", # "Stuff" means stuff all chunks into one prompt
    retriever=retriever
)

# 6. Ask
response = qa_chain.run("What is the grading policy?")
print(response)
```

Summary & References

Takeaway: RAG is the industry standard. Combines reasoning (LLM) with facts (DB).